GPADC 开发指南

版本号: 1.3

发布日期: 2025.7.25

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2023.4.24	XAA0312	初始版本
1.1	2024.2.23	XAA0311	添加 AI985 平台说明
1.2	2025.5.26	AWA2231	1. 添加 MR153 平台说明 2. 介绍 gpadc 不同的工作模式 3. 介绍接口使用与测试用例
1.3	2025.7.25	AWA2231	添加 F135 平台说明

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 文档简介 **1**
1.1 目标读者 1
1.2 适用范围 1
1.3 相关术语介绍 1

2 模块功能介绍 **2**
2.0.0.1 PWM 联动触发功能 3

3 模块配置介绍 **5**
3.1 kernel menuconfig 配置说明 5
3.2 源码结构介绍 5

4 模块接口说明 **6**
4.1 GPADC 初始化接口 6
4.2 GPADC 通道配置接口 6
4.3 GPADC 通道取消配置接口 6
4.4 GPADC 注册回调接口 6
4.5 GPADC 读取 ADC 接口 7
4.6 GPADC 去初始化接口 7
4.7 接口使用 7
4.7.0.1 代码应用流程 7
4.8 测试用例介绍 8
4.8.0.1 **测试用例说明** 8
4.8.0.2 **测试代码解析** 8
4.8.0.3 Burst 模式 9
4.8.0.4 PWM 联动触发 12

5 FAQ **15**
5.0.0.1 Q1. 当怀疑 GPADC 读的电压不准确/发生数据丢失时 15
5.0.0.2 Q2. 当 GPADC 的采样率达不到预设的值时/调用回调函数的时间过长 15

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 文档简介

介绍rtos中gpadc的使用方法，为gpadc设备驱动开发提供参考。

1.1 目标读者

gpadc驱动的开发/维护人员。

1.2 适用范围

Table: 适用产品列表

- MR527
- A523
- AI895
- MR153
- F135

1.3 相关术语介绍

术语	解释说明
sunxi	指 Allwinner 一系列 soc 硬件平台
GPADC	即 General-Purpose Analog-to-Digital Converter，通用型模拟-数字转换器，其主要功能是将连续变化的模拟信号转换为离散的数字信号。常用于电池电压监测、密钥检测、过流检测、湿度传感器等场景。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块功能介绍

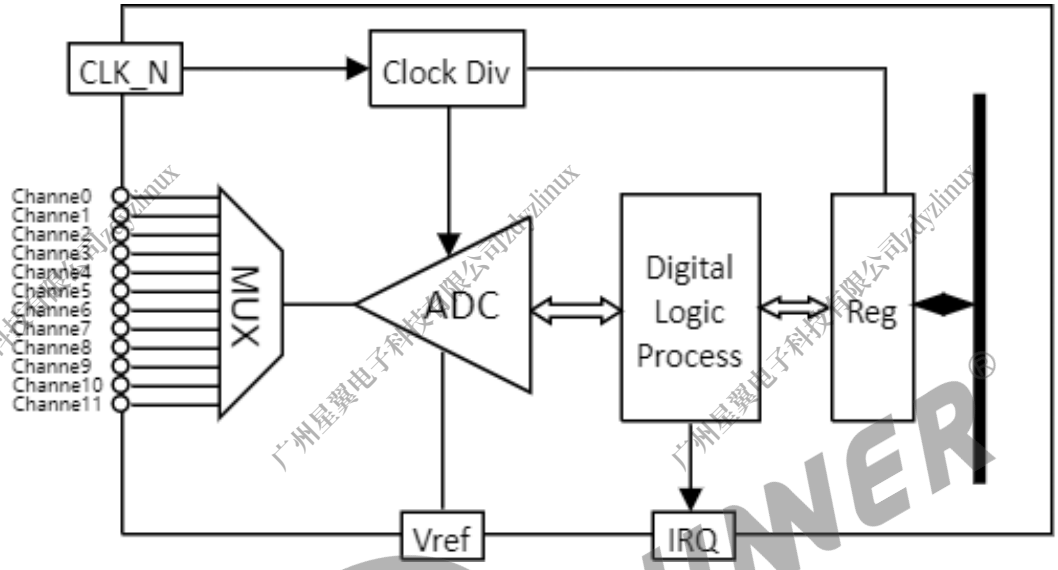

图 2-1: GPADC 硬件框图

GPADC 是 12bit 采集精度的模数转换模块，支持 24 路通道，模拟输入范围 0～1.8V，最高采样率 1MHZ，并且支持数据比较，自校验功能，同时工作于可配置的四种工作模式：

1.**Single 模式**：单次采样模式。在以此模式使能 GPADC 后，在指定通道进行一次单次采样，**在完成一次采样后就停止采样**，并将结果保存在对应通道的数据寄存器 GP_CH(x)_DAT(其中 (x) 为对应通道号) 中。

（2）Single-cycle mode：在指定的通道完成一个周期转换并将数据放在响应数据寄存器中；

（3）**Continue 模式**：连续采样模式。在以此模式使能 GPADC 后，对指定通道进行连续采样 (直到关闭 ADC 使能)，并将结果更新在对应通道的数据寄存器中，每次读取 GP_CHx_DAT(其中 (x) 为对应通道号) 寄存器都会读取到最新的值。

（4）**Burst 模式**：连续采样模式。在**使能后会一直以设置好的采样率对指定通道连续采样** (直到关闭 ADC 使能)，并将结果存放入数据 FIFO 中，直到**满足一定数量后触发中断，并自动关闭。**

使用 Burst 需要设置触发数据量 (Triger_level)，在数据量达到设置的触发数据量时，会触发中断去调用用户注册的回调函数。同一端口下的不同通道共享同一个 FIFO，按通道顺序排列。以 MR527 为例，MR527 的每个 port 具有 64byte 深度的 FIFO，如果使能了 Port1 的 0/1/2 通道，那么 Port1 的 FIFO 中的第一个数据为 channel0 的数据，第二个数据为 channel1 的数据，第三个数据为 channel2 的数据，第四个数据为 channel0 的数据……以此类推。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

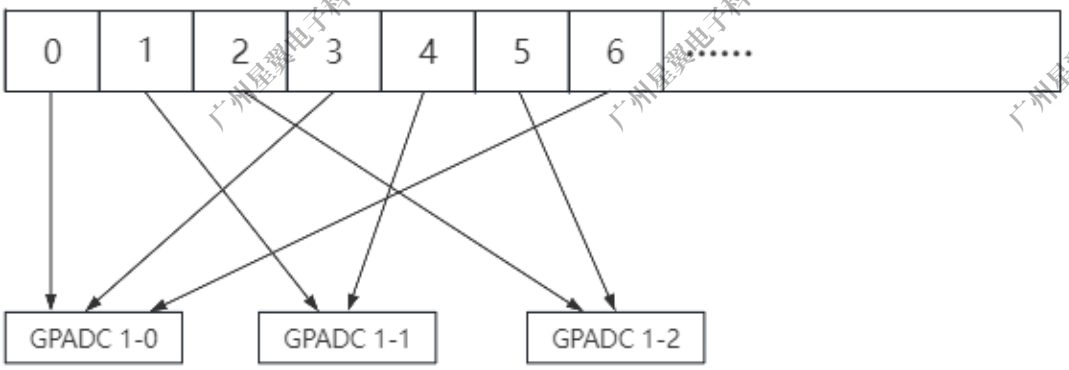

图 2-2: Burst 模式下的 FIFO 数据分配

说明

GPADC 仅在 Burst 模式下支持使用 DMA 搬运。

2.0.0.1 PWM 联动触发功能

平台	MR527	MR536	MR153
是否支持	否	是	是

在一些较新的芯片平台中支持 GPADC 与 PWM 的 pulse 模式相结合，在 PWM 设置了指定的 Pulse 个数并发送完成后，自动开启 GPADC 的采样使能位 GP_EN。

如果有必要，也允许设置在收到触发条件后延迟一段时间再触发 GPADC 的采样使能，**这个延迟时间会被驱动转化为 CLK 的个数，这个 CLK 个数必须为**

$$2^n \quad (n = 0, 1, 2, \ldots, 14)$$

（驱动中会自动选择最接近的延迟时间的满足要求的 CLK 数）。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

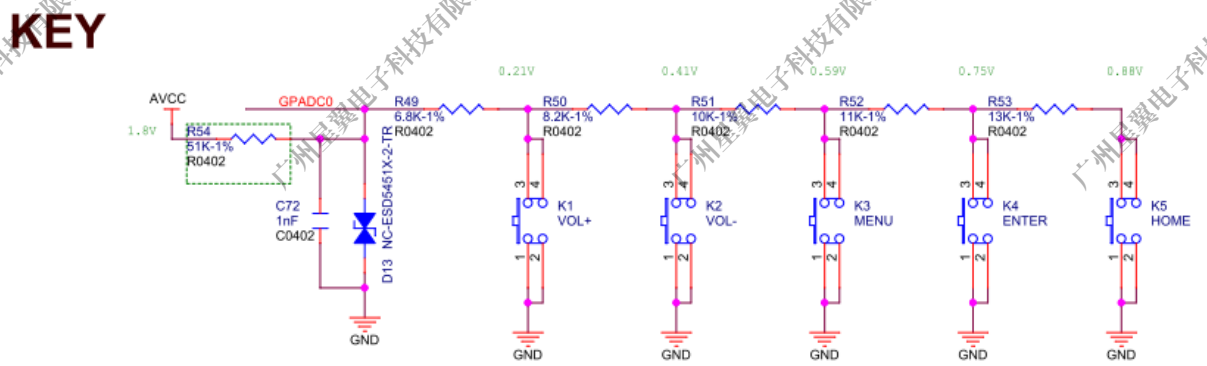

图 2-3: GPADC 典型电路

部分GPADC接口用于 KEY 模块按键的读取，一般包括 VOL+、VOL-、HOME、MENU、ENTER 等等，GPADC0 用于 KEY 的电路如上图。AVCC-AP 为 1.8V 的供电，不同的按键按下，GPADC0 口的电压不同，CPU 通过对这个电压的采样来确定具体是那一个按键按下。如上图，VOL+、VOL-、MENU、ENTER、HOME/UBOOT 对应的电压分别为 0.21V、0.41V、0.59V、0.75V、0.88V。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 模块配置介绍

3.1 kernel menuconfig 配置说明

```
.config - BitThunder Configuration
> Drivers Options > soc related device drivers > SUNXI GPADC Devices
                                                                    enabl

  There is no help available for this option.
  Symbol: DRIVERS_SUNXI_GPADC [=y]
  Type  : boolean
  Prompt: enable gpadc-ng driver
    Location:
      -> Drivers Options
        -> soc related device drivers
          -> SUNXI GPADC Devices
    Defined at drivers/rtos-hal/hal/source/gpadc-ng/Kconfig:3
```

图 3-1: menuconfig 配置

3.2 源码结构介绍

GPADC 驱动的相关源代码位于内核drivers/hal/source/gpadc-ng目录下：

```
source/gpadc-ng/
├── common_gpadc.h
├── hal_gpadc_ng.c
├── Kconfig
├── Makefile
├── platform
│   └── gpadc_sun55iw3.h
│   └── gpadc_sun55iw6.h
│   └── gpadc_sun8iw22.h
└──
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 模块接口说明

4.1 GPADC 初始化接口

- 函数原型：int hal_gpadc_init(int port);
- 功能描述：初始化指定gpadc模块，使能相应资源
- 参数说明：port：指定gpadc id
- 返回值：0：成功; 非 0：失败

4.2 GPADC 通道配置接口

- 函数原型：hal_gpadc_status_t hal_gpadc_channel_init(int port，hal_gpadc_channel_t channal, bool keyadc)
- 功能描述：初始化指定gpadc的gpadc通道
- 参数说明：port：指定需要初始化的gpadc chennel：指定初始化的channel keyadc：表明当前是否需要gpadc做按键，是传参为 1，若不是，请传 0；
- 返回值：0：成功; 非 0：失败

4.3 GPADC 通道取消配置接口

- 原型：hal_gpadc_status_t hal_gpadc_channel_exit(int port，hal_gpadc_channel_t channal)
- 功能：取消 GPADC 某个通道配置
- 参数：port：指定gpadc_id channel：通道号
- 返回值：0: 成功; 非 0：失败

4.4 GPADC 注册回调接口

- 原型：hal_gpadc_status_t hal_gpadc_register_callback(int port，hal_gpadc_channel_t channal, gpadc_callback_t user_callback)
- 功能：向应用层提供注册回调接口的功能
- 参数：port：指定gpadc_id channel：通道号 user_callback：应用层回调接口
- 返回值：0: 成功; 非 0：失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.5 GPADC 读取 ADC 接口

- 函数原型：uint32_t gpadc_read_channel_data(int port, hal_gpadc_channel_t channel)
- 功能描述：读取adc数值
- 参数说明：port：指定gpadc id channel：指定adc通道
- 返回值：0：成功; 非 0：失败

4.6 GPADC 去初始化接口

- 原型：hal_gpadc_status_t hal_gpadc_deinit(int port)
- 功能：GPADC 模块去初始化
- 参数：指定deinit哪个接口
- 返回值：0: 成功; 非 0：失败

4.7 接口使用

4.7.0.1 代码应用流程

步骤 1：初始化 GPADC 端口，对硬件资源和软件内部参数进行初始。

```
hal_gpadc_init(port);
```

步骤 2：配置 GPADC 的相关参数

```
gpadc_config->channel_used = GP_CH0_SELECT | GP_CH1_SELECT | GP_CH2_SELECT;
/* 设置gpadc的工作模式*/
gpadc_config->mode = GP_BURST_MODE;
gpadc_config->data_num_per_ch = 1200;
gpadc_config->use_dma = true;
gpadc_config->sample_rate = 2000000;

hal_gpadc_config(port, gpadc_config);
```

步骤 3：注册 GPADC 的 callback

```
hal_gpadc_register_callback(port, channel, callback, para);
```

步骤 4：使能 GPADC

```
hal_gpadc_start(port);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.8 测试用例介绍

4.8.0.1 测试用例说明

在选上对应的驱动、应用 demo 后便可使用 GPADC 来读取电压，在 RV 端开启并打印所有 GPADC port 的通道数据，使用命令 **hal_gpadc_test -a**。

```
cpu0>hal_gpadc_test
Usage: hal_gpadc_test [-a] open all port

                [-p][port_num] open GPADC port

                [-c][channel_used_bit] enable channel by bit

eg: hal_gpadc_test -p 0 -c 0x0ff


cpu0>hal_gpadc_test -a
gpadc cb thread enter


cpu0> |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 190| 503| 377| 280| 209|1341|1342| |port-2| 1016|1341|1342|1341|1342|1342|1342|1342|1341|1341| |port-3|  220
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 190| 503| 377| 280| 209|1341|1342| |port-2| 1017|1341|1341|1341|1342|1342|1342|1341|1341|1341| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 377| 281| 209|1341|1342| |port-2| 1016|1341|1342|1341|1342|1342|1342|1341|1341|1341| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 376| 279| 209|1341|1342| |port-2| 1016|1341|1342|1342|1342|1342|1342|1342|1341|1341| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 377| 280| 209|1341|1342| |port-2| 1016|1341|1341|1342|1342|1342|1342|1341|1342|1342| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 377| 280| 209|1339|1342| |port-2| 1016|1341|1342|1342|1342|1341|1342|1341|1341|1341| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 377| 280| 209|1341|1342| |port-2| 1016|1341|1342|1341|1342|1341|1342|1342|1342|1341| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 377| 280| 209|1341|1342| |port-2| 1017|1341|1342|1341|1342|1342|1341|1341|1342|1341| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 377| 280| 209|1341|1342| |port-2| 1016|1341|1342|1342|1342|1342|1341|1342|1342|1341| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 377| 281| 209|1341|1342| |port-2| 1016|1341|1341|1342|1342|1341|1342|1341|1341|1341| |port-3|  223|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 377| 279| 209|1339|1342| |port-2| 1016|1341|1341|1341|1342|1342|1342|1341|1341|1341| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 377| 280| 209|1339|1342| |port-2| 1016|1341|1342|1342|1342|1341|1342|1342|1342|1341| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 376| 280| 209|1341|1342| |port-2| 1016|1341|1341|1341|1342|1342|1342|1341|1341|1341| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 253| 189| 503| 377| 280| 209|1341|1342| |port-2| 1016|1341|1341|1342|1342|1342|1342|1341|1341|1341| |port-3|  222|
 |port-0|     0|    0|    0|    0|    0|    0|    0|    0|    0|    0| |port-1|  443| 331| 254| 189| 503| 377| 280| 209|1341|1342| |port-2| 1016|1341|1341|1342|1342|1342|1342|1342|1341|1341| |port-3|  222|
```

图 4-1: GPADC 读取的电压值

也可以单独使能一个端口 **hal_gpadc_test -p 1 -c 0x100**。

命令参数-p 1 选择 GPADC 控制器 1，第二个参数-c 0x100 为选择 GPADC 对应的通道，0x100 代表 bit8，即 GPADC1 通道 8；板端是 1.34V 接到 GPADC1_8，可读取到 data 为 1341mV，与输入电压一致。

4.8.0.2 测试代码解析

应用 demo 的代码路径在 rtos/lichee/rtos-hal/hal/test/gpadc/test_gpadc.c

上节 demo 输入的两个参数分别为 port（选择 GPADC 控制器）和 channel（GPADC 控制对应的通道）

使用函数步骤流程介绍如下（相关参数定义省略，详细可见代码）：

```
int sunxi_gpadc_all_test(int argc, char **argv)
{
    ……
    gpadc_config = hal_malloc(sizeof(hal_gpadc_config_t));
    memset(gpadc_config, 0 ,sizeof(hal_gpadc_config_t));
    gpadc_config->mode = GP_CONTINUOUS_MODE;
    gpadc_config->sample_rate = 1000000;
    gpadc_config->channel_used = (GP_CH0_SELECT | GP_CH1_SELECT | GP_CH2_SELECT
        | GP_CH3_SELECT | GP_CH4_SELECT | GP_CH5_SELECT | GP_CH6_SELECT
        |GP_CH7_SELECT |GP_CH8_SELECT |GP_CH9_SELECT;

    for (i = 0; i < GPADC_PORT; i++) {
        /* 初始化gpadc端口 */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    hal_gpadc_init(i);
    /* 将写好参数的结构体配置到gpadc-port */
    hal_gpadc_config(i, gpadc_config);
    /* 使能gpadc */
    hal_gpadc_start(i);
  }
  ......
  /* 创建线程，在线程中读取数据，详情请看代码文件 */
  ......
  return 0;
}
FINSH_FUNCTION_EXPORT_CMD(sunxi_gpadc_all_test, hal_gpadc_test, gpadc hal all channel tests)

/* 以下线程用于循环打印gpadc端口的值 */
void gpadc_test_thread(void *param)
{
  ......
  while(1) {
    for (; i < GPADC_PORT; i++) {
      printf(" |port-%d| ", i);
      /* 获取当前gpadc-port的配置参数(如果不知道的话) */
      hal_gpadc_get_config(i, gpadc_config);

      for (j = 0; j < GPADC_CHANNEL_MAX[i]; j++) {
          if ((gpadc_config->channel_used >> j) & 0x01) {
            /* 读取数据并打印 */
            vol_data[j] = gpadc_read_channel_data(i,j);
            printf("%4d|", vol_data[j]);
          }
      }
    }
    printf("\n");
    hal_msleep(10);
  }

  /* 反初始化gpadc，释放资源 */
  for (i = 0; i < GPADC_PORT; i++)
    hal_gpadc_deinit(i);
  hal_thread_stop(NULL);
}
```

4.8.0.3 Burst 模式

4.8.0.3.1 测试用例说明

在选上对应的驱动、应用 demo 后，在 RV 端使用 GPADC 0_8 读取对应的电压输入，在小机端可使用如下命令：

```
hal_gpadc_ng_burst 1 0x03 40 1 1000000 10000
```

得到 log 如下：

```
命令原型：
hal_gpadc_ng_burst [port] [channel_bit] [data_len] [use_dma] [sample_rate] [period_us]
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
RTOS串口控制台：
cpu0>hal_gpadc_ng_burst 1 0x03 10 1 1000000
Run gpadc test with burst mode

Linux串口控制台：
root@TinaLinux:/# cat /sys/kernel/debug/remoteproc/remoteproc0/aw_trace_log?[J
Run gpadc test with burst mode
[dma] port 1 channel 0 :
[0]1011
[1]1011
[2]1011
[3]1011
[4]1011
[5]1011
[6]1011
[7]1011
[8]1011
[9]1011

[dma] port 1 channel 1 :
[0]2040
[1]2039
[2]2039
[3]2039
[4]2040
[5]2040
[6]2039
[7]2039
[8]2039
[9]2039
```

输入参数作用如下：

参数名	功能介绍
port	GPADC 控制器 ID（MR536 有 4 个 GPADC 控制器，MR527 有 2 个）
channel_used	GPADC 通道（每个 GPADC 对应若干个通道，详细参考用户手册）
data_num_per_ch	每个通道采集的数据长度，即每个通道采样多少个数据之后再一起触发 port 中断。
use_dma	本次采样是否使用 dma
sample_rate	本次采样的采样率

上述命令中，第一个参数 0 代表选择了 GPADC 控制器 port 0，第二个参数 0x03 是位掩码 bit0 和 bit1，代表选择了 port1 的 channel 0 和 channel 1，第三个参数 20 代表着采集 20 个数据触发一次中断，在中断中读取一次数据。

在使用 burst 模式时，同一个 port 口下的所有 channel 都会被设置为 burst 模式，**共享 FIFO 和 FIFO 中断**。

可以使用以下接口去读取指定 port-channel 的数据：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
hal_gpadc_fifo_read_channel_data(int port, hal_gpadc_channel_t channel,uint16_t *data_buf)
```

说明

由于共享 FIFO 中断，同一 port 口不同 channel 的 callback 由驱动分发。此处设置的 data_len 是单个 channel 的 data_len，即如果开启了 2 个通道，data_len 设置为 10，则当每个通道采样到 10 个数据，共计 20 个数据时则触发 Port 的中断，按顺序调用注册 channel 的 callback。

4.8.0.3.2 测试代码解析

使用函数步骤流程介绍如下（相关参数定义省略，详细可见代码）：

```
int sunxi_gpadc_burst_test(int argc, char **argv)
{
    int ret = -1;
    struct gpadc_test_para_t gpadc_para;

    //初始化默认参数
    /* 配置端口的采样率 */
    gpadc_para.cfg.sample_rate = DEFAULT_SR;
    /* 配置Port模式：初始化为Burst模式 */
    gpadc_para.cfg.mode = GP_BURST_MODE;
    /* 配置是否使用DMA */
    gpadc_para.cfg.use_dma = true;
    /* 配置使能的通道 */
    gpadc_para.cfg.channel_used = GP_CH0_SELECT | GP_CH1_SELECT;

    /* 初始化GPADC的硬件端口 */
    ret = hal_gpadc_init(gpadc_para.port);

    /* 将写好的参数配置进port */
    hal_gpadc_config(gpadc_para.port, &gpadc_para.cfg);

    /* 申请读取数据的buffer */
    gpadc_para.data_buf = hal_malloc(gpadc_para.cfg.data_num_per_ch
                    * GPADC_CHANNEL_MAX[gpadc_para.port] * sizeof(uint16_t));

    /* 注册中断的callback */
    hal_gpadc_register_callback(gpadc_para.port, gpadc_para.channel,
            sunxi_gpadc_irq_callback_for_channel, &gpadc_para);

    /*使能gpadc，开始采样*/
    hal_gpadc_start(gpadc_para.port);

    return 0;
}
FINSH_FUNCTION_EXPORT_CMD(sunxi_gpadc_burst_test, hal_gpadc_ng_burst, gpadc hal APIs tests)

/* 采样结束触发中断时调用的回调函数 */
static int sunxi_gpadc_irq_callback_for_channel(void *param)
{
    ......
    /* 如果传参没有传入channel_used等配置参数，可以使用这个函数去获取
    // hal_gpadc_config_t cfg;
    // hal_gpadc_get_config(gpadc_para->port, &cfg);
     */

    /* 判断该通道是否使能 */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
  for (i = 0; i < GPADC_CHANNEL_MAX[gpadc_para->port]; i++) {
    if (!(gpadc_para->cfg.channel_used & (0x01 << i)))
      continue;
    /* 读取数据 */
    len = hal_gpadc_fifo_read_channel_data(gpadc_para->port,
          i, gpadc_para->data_buf);

    /* 处理数据：此处将数据打印出来 */
    amp_printf("%s port %d channel %d :\n",
          gpadc_para->cfg.use_dma ? "[dma]" : "[fifo]", gpadc_para->port, i);
    for (j = 0; j < len; j++)
      amp_printf("[%d]%u \n", j, *(gpadc_para->data_buf + j));
    amp_printf("\n");
  }
  ......
  return 0;
}
```

4.8.0.4 PWM 联动触发

4.8.0.4.1 测试用例说明

测试用例代码路径如下：

```
rtos/lichee/rtos-hal/hal/test/pwm_gpadc/test_pwm_gpadc.c
```

除了 GPADC 驱动及 demo 需要使能外，还需要打开 PWM 的驱动。

```
BitThunder Configuration
 └──> Drivers Options
   └──>soc related device drivers
     └──>SUNXI GPADC Devices
     └──>SUNXI PWM Devices
       └──> [*] enable pwm-ng driver
```

在选上对应的驱动、应用 demo 后编译烧录即可。

该测试用例使用 PWM1-0 触发 GPADC2-0，在配置完成 PWM 的发送指定的 9 个周期波形后，延时 170us 自动使能 GPADC2-0 的 burst 模式进行采集，采集完毕返回一个 callback。

在小机端输入如下命令：

```
cpu>hal_pwm_triger_gpadc
```

得到 log 如下：

```
RV端：
uart>hal_pwm_triger_gpadc
gpadc cb thread enter

Linux端：
cat /sys/kernel/debug/remoteproc/remoteproc0/aw_trace_log
gpadc cb thread enter
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[dma] port 2 channel 0 :
[0]186
[1]184
[2]182
[3]180
[4]178
[5]175
[6]173
[7]172
[8]169
[9]167
[10]165
[11]164
……
……
[1190]3920
[1191]3923
[1192]3927
[1193]3928
[1194]3930
[1195]3932
[1196]3935
[1197]3937
[1198]3939
[1199]3941
```

4.8.0.4.2 测试代码解析

使用函数步骤流程介绍如下（相关参数定义省略，详细可见代码）：

```
int sunxi_triger_gpadc_test(int argc, char **argv)
{
    ……

    /* 配置PWM的相关参数pwm_cfg */
    pwm_cfg = hal_malloc(sizeof(struct pwm_config));
    ……
    hal_pwm_init(pwm_port); /*初始化PWM端口*/

    /* 配置GPADC的相关参数gpadc_para */
    gpadc_para->port = 2;
    gpadc_para->channel = 0;
    gpadc_para->delay_ns = 168000;    //在PWM发送完毕后间隔168us触发GPADC采样
    gpadc_para->cfg.channel_used = GP_CH0_SELECT;
    gpadc_para->cfg.mode = GP_BURST_MODE;
    gpadc_para->cfg.data_num_per_ch = 1200;
    gpadc_para->cfg.use_dma = true;
    gpadc_para->cfg.sample_rate = 2000000;

    ……
    /* 初始化GPADC */
    ret = hal_gpadc_init(gpadc_para->port);
    if (ret) {
        hal_log_err("[gpadc%d] init failed!\n" , gpadc_para->port);
        return -1;
    }
    /* 初始化相关配置 */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    hal_gpadc_config(gpadc_para->port, &gpadc_para->cfg);

    /* 注册一个dma的callback函数，在完成数据接收时触发 */
    hal_gpadc_register_callback(gpadc_para->port, gpadc_para->channel, sunxi_gpadc_callback, gpadc_para);
    /* 设置GPADC为外部信号（PWM）触发模式，并设置触发延时 */
    hal_gpadc_external_triger_set(gpadc_para->port, gpadc_para->delay_ns);

    hal_gpadc_start(gpadc_para->port);
    /* 设置PWM的外部触发 */
    hal_pwm_triger_gpadc_enable(pwm_port, pwm_channel);
    /* 配置PWM并使能输出 */
    hal_pwm_control(pwm_port, pwm_channel, pwm_cfg);

    hal_msleep(3000);

    /* 释放相关资源 */
    hal_pwm_release(pwm_port, pwm_channel, pwm_cfg);
    hal_pwm_deinit(pwm_port);
    hal_gpadc_deinit(gpadc_para->port);
    hal_free(gpadc_para->data_buf);
    hal_free(pwm_cfg);

    return 0;
}

static int sunxi_gpadc_callback(void *param)
{
    struct gpadc_test_para_t *gpadc_para = (struct gpadc_test_para_t*) param;
    /* 回调函数中读取数据 */
    len = hal_gpadc_fifo_read_channel_data(gpadc_para->port, gpadc_para->channel, gpadc_para->data_buf);

......
    return 0;
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 FAQ

5.0.0.1 Q1. 当怀疑 GPADC 读的电压不准确/发生数据丢失时

A1：① 使用万用表/示波器测量输入电压源，并且使用 Max/Min 功能记录峰-峰值，确认是否由于输入电压抖动引起的误差。

② 如果使用的是 Burst 模式，可以在驱动中打开 CONFIG_DRIVERS_GPADC_DEBUG 宏，使用 **gpadc_fifo_debug [port]** 命令，该命令可以打印必要的参数，包括寄存器、软件 buffer 中的数据等，确认是丢失数据还是多出数据。

③ 将指定 Port 下功能的相关寄存器 dump 出来（方法参考《读写寄存器》），检查时钟源、模式、采样率和采样时间是否与设置的一致，且是否在合理范围内。具体配置关系可以查看对应《User Manual》GPADC 章节下对应的《Clock and Timing Requirement》小节，通常情况下，采样时间 TACQ 的值不应小于 0x02。

5.0.0.2 Q2. 当 GPADC 的采样率达不到预设的值时/调用回调函数的时间过长

A2：① 可以接入信号发生器，抓取一个周期内规律变化的波形。将采样数据和实际波形对比，定位是采样时间过长还是采样完成但 callback 没有被立刻调到。

如果是前者，参考问题回答 A1 的点 ② 和点 ③；如果是后者，可以使用 **list_irq** 命令排查是否有其他中断同时触发，再排查是否其他中断的处理时间过长。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。